

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento de Aprendizaje Continuo (AC)

Proyecto I — Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

Integrantes

Randall Bolaños López	2019043784
Aldo Alessandro Bagliano Gómez	2020018456
Yendry Badilla Gutierrez	2016113047
Christopher David Rodriguez Cordero	2022040771

Profesor

Dr.-Ing. Jeferson González Gómez

Octubre de 2026

Índice

1. Introducción	2
2. AC1 — Necesidades de aprendizaje	2
2.1. Contexto	2
2.2. Conocimientos	2
2.3. Habilidades y destrezas	3
2.4. Actitudes	3
3. AC2 — Tecnologías nuevas y emergentes	3
4. AC3 — Estrategias implementadas	4
4.1. Ir a la fuente primaria	4
4.2. Construir de menor a mayor	5
4.3. Simular para no depender del hardware	5
4.4. Escribir para entender	5
4.5. Organizar el tiempo	5
4.6. Medir antes y después	5
4.7. Revisar entre pares y con herramientas	5
4.8. Apoyarse en asistentes de inteligencia artificial, verificando	6
4.9. Búsqueda bibliográfica	6
5. AC4 — Evaluación crítica de las estrategias	6
5.1. Lo que el grupo haría distinto	7
5.2. Balance	7
6. Conclusiones	7

1. Introducción

Este documento evidencia el atributo de **Aprendizaje Continuo (AC)** en el Proyecto I. Identifica lo que el grupo no sabía y necesitaba saber (AC1), las tecnologías que contribuyeron a aprenderlo (AC2), las estrategias concretas que se aplicaron (AC3) y una evaluación de qué tan bien funcionaron (AC4).

Cada afirmación se apoya en un rastro verificable del repositorio: un documento de `docs/`, un commit o un issue. El aprendizaje que aquí se describe es el que dejó evidencia.

2. AC1 — Necesidades de aprendizaje

2.1. Contexto

El proyecto pide construir un sistema operativo a la medida, una biblioteca de control, un servidor web y un modelo físico con su electrónica de potencia. Al inicio, el grupo tenía experiencia en programación en C, en Linux como usuario y en electrónica digital básica, y parte del grupo ya había trabajado con electrónica de potencia. No había construido una imagen con Yocto, ni integrado un sensor inercial, ni diseñado piezas para impresión 3D.

A eso se suma el cambio tecnológico. Las herramientas del proyecto cambian más rápido que la documentación que las explica: la sintaxis de Yocto varió entre versiones, las distribuciones del host son más nuevas que las validadas por la rama `scarthgap`, y el kernel 6.6 exige un módulo de WiFi que versiones anteriores no pedían. Buena parte de lo que se encuentra en línea está escrito para una versión que ya no aplica, y saber distinguirlo resultó ser una necesidad de aprendizaje en sí misma.

2.2. Conocimientos

Área	Qué hacía falta aprender	Por qué lo exigía el proyecto
Yocto y BitBake	Capas, recetas, <code>bbappend</code> , variables con <code>overrides</code> , <code>cache</code> <code>sstate</code> , tablas de particiones con <code>wic</code> .	La imagen mínima y la receta propia son obligatorias.
Compilación cruzada	Toolchain-SDK, <code>sysroot</code> , archivo de toolchain de CMake, verificación con <code>file</code> y <code>readelf</code> .	Nada puede compilarse en el target.
<code>systemd</code>	Unidades, dependencias (<code>Requires</code> frente a <code>Wants</code>), política de reinicio, objetivos de red.	Arranque automático con <code>Restart=on-failure</code> .
Audio en Linux embebido	ALSA, <i>device tree overlays</i> , la salida PWM de la Raspberry Pi.	Reproducir MP3 por un parlante a bordo.
Temporización de GPIO	PWM y pulsos de servo por DMA, medición de pulsos en microsegundos, I2C.	Motores, servo, HC-SR04 y MPU-6050.

Área	Qué hacía falta aprender	Por qué lo exigía el proyecto
Electrónica de potencia	Aislamiento óptico, separación de tierras, tiempos de conmutación de un optoacoplador, caída de un puente bipolar.	La nota de seguridad del enunciado es de lectura obligatoria.
Alimentación portátil	Rango de tensión de una celda de Li-Ion, función del BMS, comportamiento de una batería bajo la corriente de un motor.	Fuente portátil sin dañar el equipo prestado.
Estimación de estado	Integración de un acelerómetro y su deriva, filtro complementario, corrección de velocidad cero.	Odometría y mapa sin encoders.
Servidor HTTP en C	<code>libmicrohttpd</code> , sesiones, resumen de contraseñas.	Control remoto con autenticación.
Diseño mecánico	Modelado de las piezas para impresión 3D, montaje de los motores y las llantas, distribución del peso.	El modelo físico tiene su propio rubro en la evaluación.

2.3. Habilidades y destrezas

- **Diagnosticar sin pantalla.** El robot no tiene monitor ni teclado: todo se depura por la consola serie, por SSH y con `journalctl`.
- **Leer registros largos de construcción** y encontrar la línea que importa entre miles.
- **Medir antes de conectar.** Usar el multímetro como paso obligatorio, no como último recurso.
- **Soldar y montar** una etapa de potencia en placa perforada.
- **Trabajar con un flujo de Git profesional:** ramas, Pull Requests, Conventional Commits, issues e hitos.

2.4. Actitudes

- **Desconfiar del mensaje de error.** Varias veces el mensaje apuntaba a una causa falsa.
- **Paciencia con ciclos lentos.** La primera construcción de la imagen tomó más de cinco horas; un error de configuración se paga caro.
- **Responsabilidad sobre equipo ajeno.** Un cable mal puesto destruye una tarjeta que no es del grupo.
- **Documentar mientras se trabaja,** no al final.

3. AC2 — Tecnologías nuevas y emergentes

Estas son las tecnologías que el grupo no había usado antes y que contribuyeron al aprendizaje. Para cada una se indica qué permitió aprender.

Tecnología	Cómo contribuyó al aprendizaje
Yocto Project, versión 5.0 (scarthgap)	Obligó a entender de qué está hecho un sistema Linux: cada paquete de la imagen es una decisión explícita. Justificar cada uno (docs/paquetes.md) fue la mejor forma de aprender qué hace cada uno.
Emulación con QEMU	En modo usuario (qemu-aarch64) permitió ejecutar los binarios ARM en la laptop. En modo sistema, con una máquina propia (qemuarm64-robot), permitió arrancar la imagen completa y observar systemd sin la Raspberry Pi.
Simulación del hardware	Un reemplazo de pigpiod que mueve un robot virtual (sim/). Escribirlo exigió entender con precisión qué hace cada llamada de GPIO, y permitió experimentar con la navegación sin riesgo.
Información de presión del kernel (PSI)	/proc/pressure y las variables BB_PRESSURE_MAX_* de BitBake: una forma reciente de regular la carga por la presión real de memoria, en lugar de un número fijo de hilos.
Sensores inerciales MEMS	El MPU-6050 llevó a estudiar fusión de sensores con un caso real y medible.
pigpio con DMA	Mostró cómo generar temporización estable desde espacio de usuario en un sistema que no es de tiempo real.
Amplificación clase D	El PAM8403 frente al LM386: el mismo problema resuelto con una fracción del consumo.
Device tree overlays	audremap mostró que redirigir un periférico a otro pin es configuración, no código.
GitHub Issues, hitos, tablero de proyecto y la CLI gh	Hicieron visible el estado del proyecto para los cuatro integrantes y dejaron el rastro de cada decisión.
LaTeX con TikZ y CircuiTikZ	Los diagramas de hardware viven como texto en el repositorio y se revisan como cualquier otro cambio.
Asistentes de inteligencia artificial generativa	Se usaron en cuatro tareas: generar los diagramas, apoyar el diseño de las piezas 3D, redactar la documentación y diagnosticar los fallos de la construcción con Yocto. Acortaron el camino entre un mensaje de error y su causa, y permitieron probar más variantes de un diseño en menos tiempo.

Cómo se verificó lo que produjo la inteligencia artificial. Nada de lo que propuso un asistente se dio por bueno sin comprobarlo. Los arreglos de Yocto se validaron construyendo la imagen y probándola en la Raspberry Pi; los diseños 3D, imprimiéndolos, montándolos en el robot y corrigiéndolos; y la documentación y los diagramas, leyéndolos entre los integrantes del grupo contra el robot real.

4. AC3 — Estrategias implementadas

4.1. Ir a la fuente primaria

Ante un problema, la primera consulta fue la documentación oficial, la hoja de datos o el código fuente, antes que un foro.

El caso más claro fue el fallo de descarga del kernel. BitBake reportaba que una revisión no

existía en la rama. Con ayuda de un asistente de inteligencia artificial se rastreó la causa hasta el código del *fetcher*: ejecuta `git branch -contains` descartando la salida de error, y las versiones recientes de git abortan ese comando en el repositorio espejo. El grupo comprobó la solución construyendo la imagen. El mensaje era falso; la causa y la solución quedaron documentadas en `docs/yocto-setup.md`.

4.2. Construir de menor a mayor

Cada capa se validó antes de montar la siguiente: primero `core-image-minimal`, para comprobar el BSP, el WiFi y el audio; luego la imagen del proyecto. En el hardware se siguió el mismo orden, con los procedimientos de verificación de `docs/hardware-aislamiento.md` y `docs/hardware-alimentacion.md`: aislamiento sin energía, dominio de potencia solo, optoacopladores con fuente de banco, dominio lógico solo, y la Raspberry Pi de última.

4.3. Simular para no depender del hardware

El hardware llegó después que el software. Para no detener el aprendizaje se construyeron tres ambientes sin la Raspberry Pi: la biblioteca contra el simulador en la laptop, el binario ARM bajo `qemu-aarch64`, y la imagen completa en QEMU. El programa `sim/prueba_librobot` ejercita cada módulo con 47 comprobaciones.

4.4. Escribir para entender

Cada decisión de diseño se documentó en el momento de tomarla, con la alternativa descartada y el motivo. La carpeta `docs/` tiene catorce documentos técnicos escritos durante el desarrollo, no después. Explicar por escrito por qué $390\ \Omega$ y no $220\ \Omega$ obliga a hacer la cuenta.

4.5. Organizar el tiempo

- El enunciado se desglosó en 37 issues agrupados en 10 hitos con fecha, de *Repositorio y flujo de trabajo* (8 de setiembre) a *Documentación y entrega final* (6 de octubre).
- Un tablero de proyecto con tres columnas mostró en todo momento qué estaba pendiente, en curso y terminado.
- El trabajo se repartió por áreas entre los cuatro integrantes.
- Cada integrante trabajó en su rama; los cambios entraron a `develop` y a `main` por Pull Request (más de 20 integrados).

4.6. Medir antes y después

Los problemas de desempeño se trataron como experimentos. El servidor arrancaba a los 130 s; se identificaron dos causas independientes en la configuración de red, se corrigieron y se volvió a medir: 17 s. El commit registra la medición anterior, la causa y la medición posterior.

4.7. Revisar entre pares y con herramientas

Los Pull Requests fueron el punto de revisión entre integrantes. Se sumaron verificaciones automáticas donde existían: `systemd-analyze verify` detectó una directiva ubicada en la sección equivocada que `systemd` ignoraba sin avisar.

4.8. Apoyarse en asistentes de inteligencia artificial, verificando

Los asistentes de inteligencia artificial se usaron como un compañero de consulta: para interpretar un fallo de la construcción con Yocto, para proponer un diagrama o una pieza, o para redactar un documento. La regla fue que la respuesta es una hipótesis: se compila, se imprime o se lee antes de aceptarla.

4.9. Búsqueda bibliográfica

Las hojas de datos del PC817, el L298N, el MPU-6050 y el PAM8403 fueron la base de los cálculos de `docs/hardware-*.md`: corriente del LED del optoacoplador, relación de transferencia de corriente, caída del puente, ganancia del amplificador.

5. AC4 — Evaluación crítica de las estrategias

Estrategia	Qué funcionó	Qué no funcionó
Ir a la fuente primaria	Resolvió problemas que ninguna búsqueda habría resuelto, como el del <i>fetcher</i> . Lo aprendido se sostiene al cambiar de versión.	Es lenta. Llegar hasta el código de BitBake tomó horas, aun con ayuda de un asistente de inteligencia artificial.
Construir de menor a mayor	Cada fallo quedó acotado a la última capa agregada.	Postpone los problemas de integración: los que solo aparecen con todo junto se descubren tarde. Tampoco evitó perder una tarjeta: una Raspberry Pi dejó de funcionar durante el desarrollo, se presume que por su antigüedad, y el profesor la reemplazó.
Simular	El software avanzó semanas antes de tener el robot: biblioteca, servidor, navegación y mapa se probaron sin hardware.	El simulador solo contiene lo que el grupo supo modelar. No modela el tiempo de apagado del optoacoplador, y por eso la PWM de 1 kHz funcionó en la simulación y no movió los motores reales.
Escribir para entender	Detectó errores antes de armar: la inversión del optoacoplador se encontró al documentar el circuito, no al encenderlo.	Cada cambio de diseño obligó a actualizar varios documentos. Cuando el radar reemplazó a los tres sensores, parte de la documentación quedó atrasada un tiempo.
Organizar el tiempo	El tablero evitó trabajo duplicado y dejó claro qué faltaba.	Las fechas de los hitos fueron optimistas: cinco vencieron con issues abiertos. No se previó cuánto dependía cada hito de tener el hardware armado.
Medir antes y después	Convirtió una impresión en un dato: 130 s a 17 s.	Las métricas de eficiencia se dejaron para el final, cuando ya había poco margen para actuar sobre ellas.

Estrategia	Qué funcionó	Qué no funcionó
Revisión entre pares	Los Pull Requests se revisaron antes de integrarse y dejaron el rastro de cada cambio.	La revisión del código no alcanza lo que solo aparece en el hardware: el problema de la PWM pasó todas las revisiones y se descubrió en la placa.
Asistentes de inteligencia artificial	Aceleraron el diagnóstico de los fallos de Yocto, los diagramas, las piezas 3D y la documentación.	No ahorran la comprobación: cada respuesta hubo que compilarla, imprimirla o leerla, y eso también toma tiempo.

5.1. Lo que el grupo haría distinto

1. **Encender el hardware antes.** Una prueba mínima de un motor a través de un opto-acoplador en la primera semana habría mostrado el problema de la PWM con un mes de margen. La simulación complementa a la placa, no la reemplaza.
2. **Aclarar las ambigüedades del enunciado al inicio.** La decisión de cubrir las detecciones frontal y lateral con un solo sensor sobre un servo dependía de una interpretación. El profesor la aceptó, igual que la velocidad fija de los motores, pero convenía consultarlo antes de diseñar alrededor de ella y no al final.
3. **Cerrar el diseño mecánico antes de comprar.** El chasis se rehízo, y con él hubo que cambiar los motores y descartar los primeros. Un prototipo temprano de la parte mecánica habría evitado esa compra.
4. **Preparar el host de construcción desde el primer día.** Los cuelgues por falta de memoria costaron días que una máquina adecuada, o la regulación por presión desde el inicio, habrían evitado.
5. **Medir de forma continua.** Tomar las métricas con cada imagen permite reaccionar; tomarlas al final solo permite reportarlas.
6. **Poner fechas a los hitos según sus dependencias,** no según el calendario.

5.2. Balance

Las estrategias cumplieron su propósito: el grupo terminó el proyecto dominando herramientas que al inicio no conocía, y lo aprendido quedó escrito de forma que otro grupo podría reproducirlo. Lo que más costó fueron tres cosas: Yocto, por sus fallos de construcción y sus ciclos largos; el diseño mecánico, que hubo que rehacer; y la electrónica, hasta lograr que la alimentación y los motores movieran el robot. La lección que se repite es que las estrategias que más enseñaron —leer la fuente, documentar, simular— son también las más costosas en tiempo, y que ninguna sustituye el contacto temprano con el hardware real.

6. Conclusiones

- Las necesidades de aprendizaje más grandes no estaban en la programación sino en lo que la rodea: construir el sistema operativo, aislar la electrónica de potencia y diagnosticar sin pantalla.
- La emulación y la simulación fueron las tecnologías que más contribuyeron, porque permitieron aprender sin esperar al hardware y sin arriesgarlo.

- Aprender en un entorno que cambia exige ir a la fuente primaria y verificar la versión de todo lo que se lee.
- La evaluación crítica muestra un límite claro: lo que no se modela no se aprende hasta tocar el hardware.